



英雄体系搭配平台离线迁移实现提示词

将下面的提示词直接粘贴到新的 Codex 任务中运行 °它要求以 Goal 形式持续执行 ’最终交付可运行 `可维护 `完全离线的 macOS 应用 ’同时保留未来迁移到 Windows 的能力 °

实现提示词

请创建一个 Goal，并持续执行到目标真正完成。不要只输出方案、示例代码或脚手架；需要在当前工作区完成实现、测试

Goal 目标：

将 <https://cq-zys.cn/hero-lineup> 的“英雄体系搭配平台”重建为一个完全离线、可维护、由 Rust 包装的桌面应用。

当前只要求在 macOS 上完成开发、测试和构建，但工程架构、路径处理、数据格式和前端代码必须兼容未来的 Windows

背景：

- 原项目由外包团队开发，没有源代码。
- 当前工作区可能为空。
- 线上可以取得编译后的 React/Vite bundle、游戏 JSON、图片和公开接口响应。
- 线上编译产物只能作为行为和样式参考，不得作为最终长期运行或维护的核心源码。
- 需要重新建立完整、清晰、可测试的源码。
- 用户拥有对该工具进行迁移和重建的授权。
- 工作范围只包括 /hero-lineup 工具及其必要的本地支撑功能，不需要迁移整个 cq-zys.cn 网站。

允许：

- 使用子代理并行处理线上资源盘点、数据建模、UI 重建、算法分析和测试。
- 使用浏览器检查线上页面和交互。
- 下载并归档本工具正常运行所需的公开线上静态资源和接口样本。
- 在工作区创建完整项目、测试、构建脚本和文档。
- 安装项目所需的正常开发依赖。

技术要求：

1. 桌面框架使用 Tauri 2。
2. 前端使用 React + TypeScript + Vite。
3. Rust 负责：
 - SQLite 持久化；
 - 本地数据包管理；
 - 文件导入导出；
 - 数据完整性校验；
 - 英雄与勇士属性计算；
 - 装备限制校验；
 - 任务模拟；
 - 后台计算任务；

- 应用版本和数据版本管理。
4. SQLite 可以使用 `rusqlite`、`sqlx` 或合适的 Tauri SQLite 方案，但数据库访问应尽量由 Rust 封装，避免
 5. 任务模拟不能在 UI 主线程执行；使用 Rust 后台线程或异步任务，并向前端报告进度。
 6. 所有文件路径必须通过 Tauri/Rust 跨平台路径 API 取得，禁止硬编码 ``/Users``、盘符、反斜杠或 macOS 专
 7. 平台差异使用：
 - `tauri.conf.json`
 - `tauri.macos.conf.json`
 - `tauri.windows.conf.json`
 - 必要时使用 Rust ``#[cfg(target_os = "...")]``
 8. 禁止在核心业务代码中散布平台判断。
 9. macOS 作为当前验收平台；Windows 暂不要求实际构建，但必须提供完整构建文档和配置。

完全离线要求：

- 应用安装完成后，断开网络仍可使用全部核心功能。
- 运行时不得请求 `cq-zys.cn`、`cdn.cq-zys.cn`、Google Fonts 或任何第三方地址。
- 不建立 `Socket.IO`、`WebSocket` 或访问统计连接。
- 不依赖登录、JWT、远程 API、远程图片、远程字体或在线更新。
- 使用本地字体或系统字体。
- 所有 JSON、图片、图标、默认体系和必要数据必须随安装包提供。
- 使用测试确认运行时没有网络请求。
- macOS 测试版本不要求在线更新功能。

功能范围：

A. 体系管理

- 默认体系。
- 新建、编辑、复制、删除体系。
- 多体系切换。
- 未保存修改提示。
- 公开/私有概念在离线版中改为普通本地元数据，不产生联网行为。
- 热门体系改为“内置示例体系”或“本地模板库”。
- 所有体系保存到 SQLite。
- 应用重启后数据完整恢复。

B. 勇士阵容

- 显示全部可用勇士。
- 勇士中文名称、元素和图标。

- 勇士配装面板。
- 等级、Rank、种子、泰坦状态、卡片等级。
- Familiar、Aurasong 等装备配置。
- 装备统计。
- 拖入冒险任务。
- 配装导入导出。

C. 英雄阵容

- 添加职业并生成独立英雄实例。
- 自定义英雄名称。
- 职业排序与元素排序。
- 六个装备槽。
- 品质、元素、精魂、闪耀、超越等配置。
- 等级、种子、卡片等级和技能。
- 装备合法性校验。
- 英雄复制与删除。
- 英雄配装模板。
- 装备统计。
- 拖入冒险任务。
- 阵容图片导出。
- 配装配置导入导出。

D. 冒险任务

- 添加、删除、复制任务。
- 任务分组。
- 分组排序。
- 任务卡片拖拽。
- 地图和难度选择。
- 英雄与勇士组队。
- 最大人数和重复上阵限制。
- 元素屏障计算。
- Booster、精英怪、泰坦塔词条等模拟配置。
- 任务模拟。
- 模拟结果详情。
- 模拟结果图片导出。
- 保存模拟配置、结果及快照。
- 数据版本变化时，旧结果标记为过期。

E. 导入导出

- 设计版本化的 `.zyslineup`` 文件格式。
- 支持单个体系导出和导入。
- 支持全部用户数据备份与恢复。
- 支持剪贴板文本配置。
- 导入时校验 `schemaVersion`、`gameDataVersion` 和内容完整性。
- 导入旧版或缺失字段时提供迁移逻辑。
- Windows 和 macOS 之间的导出文件必须兼容。

数据要求：

先完整盘点并归档线上可公开取得的数据，包括但不限于：

- `classes.json`
- `heroes.json`
- `quests.json`
- `items.json`
- `skills.json`
- `levels.json`
- `qmodifiers.json`
- `texts_zh.json`
- `chestodds.json`
- `skillTreeNodes.json`
- `skillTreePoints.json`
- `items_type_dict.json`
- Sprite 图片资源
- 当前公开体系的结构样本

建立如下版本概念：

- `appVersion`
- `schemaVersion`
- `gameDataVersion`
- `simulatorVersion`
- `assetVersion`

创建内容清单，例如：

`content/`

```
manifest.json
  TextAsset/
  Sprite/
  defaults/
  schemas/
```

manifest.json 至少包含：

- 数据版本
- 生成时间
- 文件清单
- 文件大小
- SHA-256
- 数据统计
- 兼容的最低应用版本

数据维护工具：

在项目中实现独立 Rust CLI，例如 `hero-data`，至少支持：

- `hero-data validate <目录>`
- `hero-data build <输入目录> --output <数据包>`
- `hero-data inspect <数据包>`
- `hero-data diff <旧数据包> <新数据包>`
- `hero-data install <数据包>`
- `hero-data verify <数据包>`

数据包更新必须支持：

- 本地文件导入；
- 哈希校验；
- Schema 校验；
- 引用完整性校验；
- 原子安装；
- 更新失败回滚；
- 保留用户体系；
- 标记旧模拟结果为过期。

第一阶段允许游戏基础数据继续以 JSON 存储，不要为了“数据库化”而过早重构全部静态数据。用户体系、模板、设置和

算法重建要求：

- 从线上编译产物、公开数据和黑盒行为中提取规则。
- 不要只根据页面文案猜测。
- 为每类计算建立可读的 Rust 模块和单元测试。
- 属性计算至少覆盖：
 - 基础等级属性；
 - 等级曲线；
 - 种子；
 - 品质倍率；
 - 六槽装备；
 - 元素；
 - 精魂；
 - 技能；
 - 职业自带技能；
 - 神器；
 - 泰坦职业；
 - 闪耀；
 - 超越；
 - 卡片等级；
 - 勇士装备；
 - 任务环境修正。
- 模拟器需要覆盖线上已观察到的职业、勇士、任务和特殊规则。
- 模拟器支持固定随机种子，以便测试可复现。
- 默认支持 10,000 次模拟。
- 提供 1,000、10,000 等次数选项。
- UI 显示进度并允许取消。
- 输出成功率、平均回合、最小/最大回合、存活率、平均伤害和平均剩余生命。

黄金测试：

- 从线上公开体系中选择具有代表性的配置作为测试样本。
- 保存输入、线上结果、数据版本和允许误差。
- 为属性计算建立精确断言。
- 对随机模拟使用固定种子测试。
- 如果无法做到与线上模拟完全相同，需要记录已知差异、原因和影响范围。
- 不得通过放宽所有断言来掩盖计算差异。

UI 还原要求：

- 尽量还原线上 hero-lineup 的整体视觉风格。
- 保留蓝紫色视觉体系、白色卡片、圆角、阴影、分组和弹窗层级。
- 保留体系管理、勇士阵容、英雄阵容、冒险任务的主要布局顺序。
- 保留桌面宽度下的卡片密度和信息层级。
- 当前主要测试 macOS 桌面窗口。
- 至少测试：
 - 1440×900
 - 1280×800
 - 1024×768
 - 较窄窗口
 - Retina/高 DPI
- 使用截图对比线上页面。
- 不需要逐像素复制网站导航栏和全站页脚，但工具主体应高度相似。
- 对无法直接取得或授权不清晰的素材，建立可替换资源映射，不要让业务代码依赖具体 CDN URL。
- 支持深色模式不是当前必需项，除非现有页面已经依赖它。

工程质量：

- TypeScript 开启严格模式。
- Rust 代码通过 `cargo fmt` 和 `cargo clippy`。
- 前端通过 lint 和类型检查。
- 为 Rust 核心逻辑编写单元测试。
- 为 SQLite migration 编写测试。
- 为导入导出编写往返测试。
- 为主要页面流程编写端到端测试。
- 不提交密钥、Token、个人路径和临时文件。
- 不依赖线上 bundle 作为最终运行代码。
- 归档的线上 bundle 放在明确的 reference 或 research 区域，并在正式构建中排除。
- 对逆向确认的规则记录来源、置信度和测试证据。

建议工程结构：

```
/
apps/
  desktop/
    src/
    src-tauri/
crates/
  hero-domain/
  hero-simulator/
```

```
hero-data/  
hero-storage/  
content/  
  manifest.json  
  TextAsset/  
  Sprite/  
  defaults/  
  schemas/  
reference/  
  online-snapshots/  
  api-samples/  
  screenshots/  
  behavior-notes/  
tests/  
  fixtures/  
  golden/  
  e2e/  
docs/  
  ARCHITECTURE.md  
  DATA_MODEL.md  
  SIMULATOR_RULES.md  
  DATA_MAINTENANCE.md  
  OFFLINE_VERIFICATION.md  
  MACOS_BUILD.md  
  WINDOWS_BUILD.md  
  KNOWN_DIFFERENCES.md
```

可以根据工程实际情况调整结构，但必须保持领域计算、存储、数据维护和 UI 的边界清晰。

执行阶段：

阶段 1：现网保全

- 枚举并下载当前 hero-lineup 所需的全部 JS、CSS、JSON、图片和字体。
- 保存 HTTP 元数据、内容哈希和资源映射。
- 记录公开接口样本。
- 生成资源盘点报告。
- 不在最终应用中依赖远程 URL。

阶段 2：规格还原

- 建立功能清单。
- 建立页面状态模型。
- 建立体系数据模型。
- 建立属性计算规则文档。
- 建立任务模拟规则文档。
- 建立页面截图基准。
- 标出已确认、推断和未知规则。

阶段 3：工程骨架

- 初始化 Tauri 2、React、TypeScript、Vite 和 Rust workspace。
- 建立 macOS 和 Windows 平台配置。
- 建立 SQLite migration。
- 建立内容资源加载层。
- 完成最小 macOS 应用构建。

阶段 4：本地业务功能

- 完成体系管理。
- 完成英雄和勇士阵容。
- 完成配装编辑。
- 完成冒险任务和拖拽。
- 完成本地模板。
- 完成导入导出。
- 完成 SQLite 持久化。

阶段 5：计算与模拟

- 重建属性计算。
- 重建装备规则。
- 重建元素屏障。
- 重建任务模拟。
- 接入 Rust 后台任务、进度和取消。
- 建立黄金测试。

阶段 6：UI 高保真

- 对照线上截图迭代样式。
- 验证窗口尺寸和高 DPI。

- 验证弹窗、滚动、拖拽和图片导出。
- 修复 WebView 特有问題。

阶段 7：离线与发布验收

- 在禁网环境运行。
- 验证没有网络请求。
- 验证首次启动。
- 验证重启后持久化。
- 验证导入导出。
- 验证数据包升级和回滚。
- 运行全部测试。
- 构建 macOS `.app``；条件允许时生成 `.dmg``。
- 若缺少 Apple 签名凭据，仍需产出未签名测试包并记录正式签名步骤。
- 提供 Windows 构建配置和文档，但本阶段不要求实际 Windows 构建。

验收标准：

1. macOS 上能够安装或直接运行生成的应用。
2. 禁止网络后仍能正常使用全部核心功能。
3. 没有 `cq-zys.cn`、`CDN`、`Socket.IO`、统计或第三方字体请求。
4. 默认游戏数据和图片随应用提供。
5. 可以创建、编辑、保存和删除多个体系。
6. 应用重启后体系不丢失。
7. 英雄和勇士可以完成配装。
8. 英雄和勇士可以拖入冒险任务。
9. 可以完成 10,000 次任务模拟且 UI 不长时间无响应。
10. 可以查看和导出模拟结果。
11. 可以导入导出体系和完整备份。
12. 可以通过本地数据包更新游戏数据。
13. 数据更新失败能够回滚。
14. 主要页面风格与线上工具保持高度相似。
15. 所有正式运行代码都有可维护源码。
16. ``cargo test``、``cargo clippy``、前端类型检查和测试通过。
17. 提供 macOS 构建说明和 Windows 后续构建说明。
18. 不得把“创建了脚手架”或“实现了部分页面”视为 Goal 完成。

工作方式：

- 先检查工作区和现有文件，不要覆盖用户已有内容。

- 建立并持续更新实现计划。
- 每完成一个阶段都运行对应验证。
- 遇到错误先诊断并尝试合理替代方案。
- 不因为任务复杂、构建时间长或存在少量未知规则而提前结束。
- 只有在确实需要用户授权、外部凭据或无法合理推断的重大产品决策时才请求用户输入。
- 对一般 UI、目录和实现细节自行做出合理决定并继续。
- 在 Goal 完成前保持持续推进。
- 最终交付时明确列出：
 - 已实现功能；
 - 未实现或已知差异；
 - 测试结果；
 - macOS 应用路径；
 - 数据包路径；
 - 文档路径；
 - Windows 迁移准备情况。

目标完成定义

该 Goal 的完成条件是交付一个能够在 macOS 上运行 `断网可用` `拥有可维护源码和本地数据维护能力的离线桌面应用` 而不是只完成工程脚手架或部分页面。